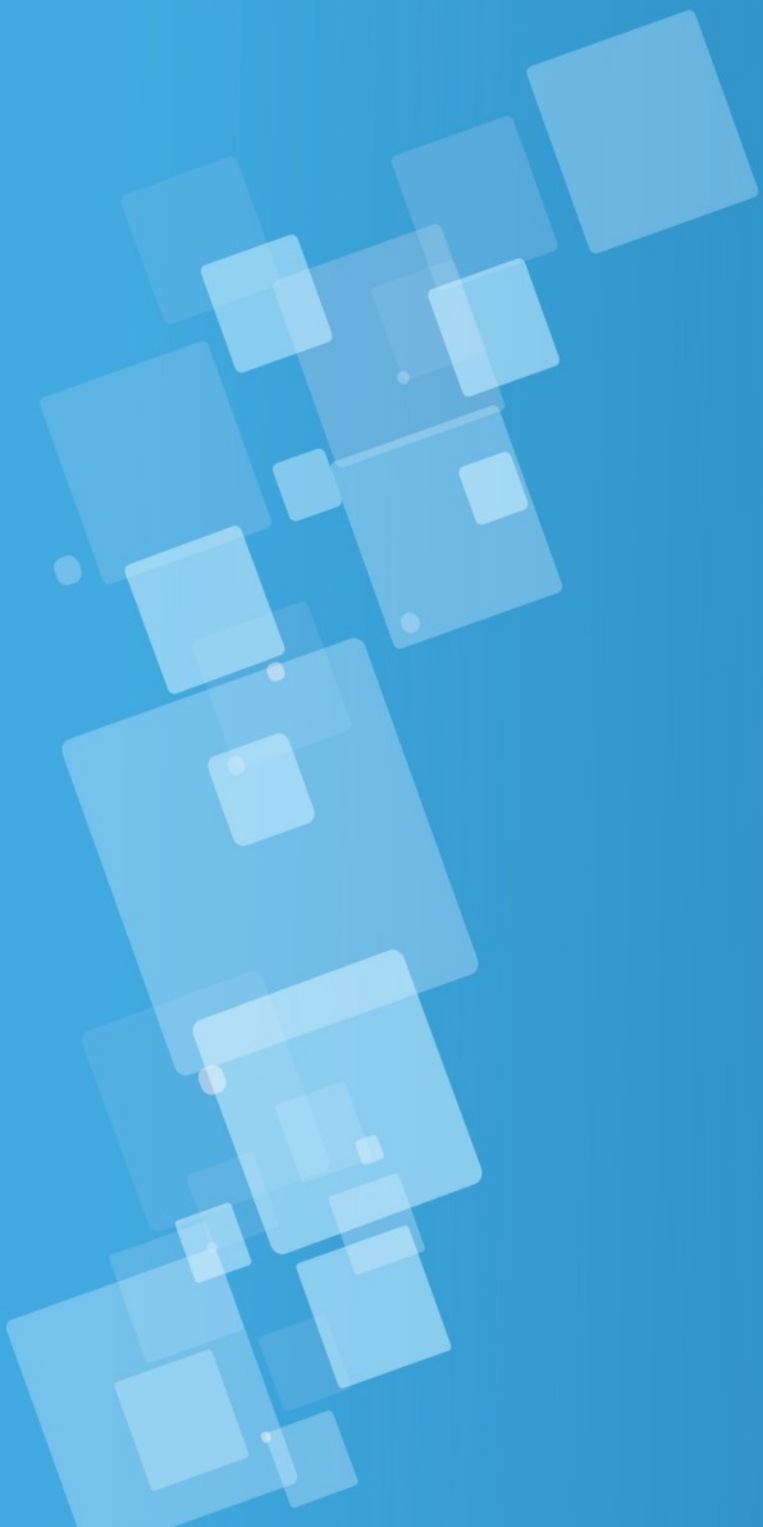




Técnicas Digitales II

RTOS (Real Time Operating System)
Sistema operativo en tiempo real
Introducción

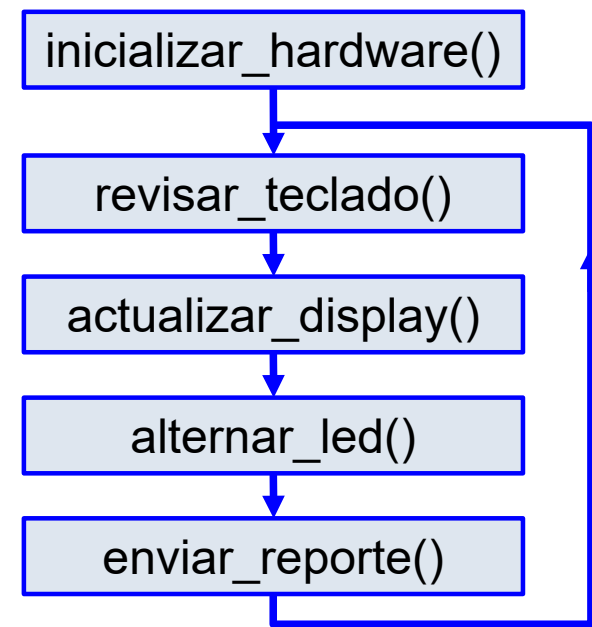
- **Bibliografía**

Mastering the FreeRTOS™ Real Time Kernel

El Método Tradicional: El "Super-Loop"

- Muchos sistemas sencillos se resuelven de esta forma:

```
void main(void) {  
    inicializar_hardware();  
    while(1) { // El "Super-Loop"  
        revisar_teclado();  
        actualizar_display();  
        alternar_led();  
        enviar_reporte();  
    }  
}
```



Este método sencillo, llama de manera secuencial a cada una de las funciones que posee.

Es útil en sistemas poco complejos, pero debe seguir reglas estrictas que pueden complicar rápidamente un desarrollo.

El Método Tradicional: El "Super-Loop"

```
void main(void) {  
    inicializar_hardware();  
    while(1) { // El "Super-Loop"  
        revisar_teclado();  
        actualizar_display();  
        alternar_led();  
        enviar_reporte();  
    }  
}
```

Las funciones deben seguir estas reglas:

- No deben ser bloqueantes.
- Deben ejecutarse rápidamente
- Deben ser divisibles (implementar Máquinas de Estado).
- Deben ser Reentrantes si son llamadas desde Interrupciones
- Los estados se deben guardar en variables globales.

No ser bloqueantes

Característica más importante.

Una función nunca debe entrar en un estado de espera activa o indefinida.

- **Qué significa:** La función no puede contener bucles que esperen un evento, ni retardos largos con bucles for o while.
- **Por qué es importante:** Detiene por completo la ejecución de todas las demás funciones del bucle.

```
void leer_sensor_bloqueante() {  
    // ... inicia la lectura ...  
    while(sensor_no_esta_listo()) {  
        // Espera activa, consumiendo 100% de CPU  
    }  
    // ... procesa el dato ...  
}
```



Deben ejecutarse rápidamente (Fast Execution)

Cada función debe completar su lógica y devolver el control al bucle principal en el menor tiempo posible.

- **Qué significa:** La función debe realizar una pequeña unidad de trabajo y terminar. El "tiempo de tick" o la latencia de todo el sistema es la suma del tiempo de ejecución de todas las funciones en una vuelta del bucle.
- **Por qué es importante:** Si las funciones tardan mucho, la capacidad de respuesta del sistema a cualquier evento se degrada.



Deben ser divisibles (implementar Máquinas de Estado)

Es una consecuencia de la anterior.

En operaciones larga, el proceso debe dividirse en pequeños pasos.

La forma clásica de lograr esto es con una máquina de estados.

- Se utiliza una variable estática (static) para recordar en qué paso (o estado) se quedó.
- **Por qué es importante:** Permite que operaciones largas (como enviar datos por un puerto serie o controlar una secuencia de motores) coexistan con otras tareas sin bloquearlas.

Deben ser divisibles (implementar Máquinas de Estado)

Ejemplo: función que lee un sensor utilizando el ADC, el proceso se inicia cuando la variable global **leer** se pone en *true*

```
void leer_sensor_no_bloqueante() {
    static enum {IDLE, INI_LECTURA, ESPERA_DATO} estado_sensor= IDLE;
    switch(estado_sensor) {
        case IDLE: // No hacer nada hasta que sea necesario
            if(leer) {
                estado_sensor = INI_LECTURA;
                leer = FALSO;
            }
            break;
        case INI_LECTURA:
            iniciar_conversion_adc(); // Operación rápida
            estado_sensor = ESPERANDO_DATO;
            break;
        case ESPERA_DATO:
            if (sensor_ya_esta_listo()) { // Polling
                procesar_dato(leer_adc());
                estado_sensor = IDLE;
            }
            break;
    }
}
```




Deben ser Reentrantes si son llamadas desde Interrupciones

En un sistema super-loop, las interrupciones siguen ocurriendo.

Si una función es llamada tanto desde el bucle principal como desde una ISR (Interrupt Service Routine), debe ser reentrante.

- **Qué significa:** La función no debe usar variables globales o estáticas de una manera que pueda ser corrompida si la función se interrumpe a sí misma. Debe trabajar principalmente con variables locales (en el stack) o parámetros.
- **Por qué es importante:** Si la ISR interrumpe el main loop justo cuando se está ejecutando una función no reentrante y la ISR llama a esa misma función, los datos pueden corromperse, llevando a un comportamiento impredecible.

Ejemplo de aplicación

Desarrollar un sistema que dado el estado de 3 llaves, encienda o apague 3 motores.

La comprobación de las llaves se debe realizar al menos 10 veces por segundo

```
void main (void)
{
    int i;
    for ( ; ; ) {
        for (i=0; i<3; i++ ) {
            if (isswitchON(i)) motorON(i);
            else motorOFF(i);
        }
    }
}
```

Cualquier procesador podrá realizar este bucle a mayor velocidad que 10 veces por segundo

- `isswitchON(i)` verifica el estado del switch `i`
- `motorON(i)` y `motorOFF(i)` enciende o apaga el motor `i`

Ejemplo de aplicación

Si ahora imponemos que los switch deben examinarse 10 veces por segundo, debemos implementar un retardo de 100 ms para luego utilizarlo en el bucle.

Podemos implementar el servicio de interrupción del timer con algo tan simple como:

```
void timerIsr (void)
{
    ticks ++;
}
```

Configuramos el timer para generar una interrupción cada 1 ms



Los “Timer”, son periféricos comunes en los microprocesadores, y pertenecen al grupo de periféricos interno del micro.

Son simple contadores que al llegar a un valor preestablecido, se reinician e interrumpen el microprocesador.

Ejemplo de aplicación

Modificamos ahora nuestra aplicación

```
void main (void)
{
    unsigned int i, delay100;
    delay100 = ticks + 100;
    for ( ; ; ) {
        if(delay100==ticks) {
            for (i=0; i<3; i++ ) {
                if (isswitchON(i)) motorON(i);
                else motorOFF(i);
            }
            delay100 = ticks + 100;
        }
    }
}
```

¿ Sería lo mismo
 $\text{delay100} \leq \text{ticks}$?

- *delay100* se inicializa con el valor actual del *tics* + 100, es decir, se adelanta al estado del tics en 100 ms (o 100 tics)
- Cuando se iguala ambas variables se realiza la verificación y se inicializa nuevamente *delay100*

Ejemplo de aplicación

Ahora sumamos otra actividad a nuestro sistema

Tenemos un sensor de presión que se debe inspeccionar cada 50ms.

- Si la presión es mayor que 100 psi se abre una válvula.
- Una vez abierta, si la presión baja de los 90 psi, se debe cerrar la válvula.

Lo presentamos como máquina de estados dentro de una función.

Sumamos otra variable *delay50* que se adelanta 50 a tics, esta variable debe ser declarada como global e inicializada en el main()

```
void checkPressure(void) {  
    if (delay50 == ticks) {  
        switch (valveState) {  
            case CLOSED:  
                if (pressure() > 100) {  
                    openValve();  
                    valveState = OPEN;  
                }  
                break;  
            case OPEN:  
                if (pressure() < 90) {  
                    closeValve();  
                    valveState = CLOSED;  
                }  
                break;  
        }  
        delay50 = ticks + 50;  
    }  
}
```

Ejemplo de aplicación

Ahora reescribimos el bucle principal como un bucle con dos funciones que son llamadas de manera secuencial.

```
unsigned int delay50, delay100;
volatile unsigned int ticks;
void main (void)
{
    iniTimer();
    delay50 = ticks + 50;
    delay100 = ticks + 100;
    for ( ; ; ) {
        checkMotorsSwitches();
        checkPressure();
    }
}
```

Agregar nuevas tareas a la aplicación, implica agregar mas funciones en la secuencia.





Ejemplo de aplicación

Finalmente, necesitamos que nuestro sistema se conecte a una red de datos.

Función *checkDatagrams*, lee los datos que ingresan, estos datos si no son leídos rápidamente se pueden perder.

Solución: Atender la lectura de datos con una interrupción, guardando los datos en una cola para luego ser leído por *checkDatagrams*

Problema: Se debe realizar un trabajo extra, implementar una cola para guardar los datos.



Limitaciones del bigloop

De lo visto se puede encontrar algunos problemas

- **Bloqueo Secuencial:** Una función lenta detiene todo el sistema.
- **Respuesta Lenta (Alta Latencia):** La rapidez con la que se detecta un evento depende del tiempo total del bucle.
En el ejemplo anterior, el bucle debe cerrar en 1 ms para que no se pase ninguna comparación de ticks.
- **Mantenimiento Complejo:** Añadir nueva funcionalidad puede romper la temporización de todo el sistema.
El código se vuelve frágil y difícil de escalar.
- **Ineficiencia:** El procesador está siempre al 100%, incluso si no hay nada que hacer.
Se desperdicia energía y tiempo de CPU preguntando "¿ya pasó algo?" (polling) .



Comparación con RTOS

El ejemplo anterior tiene tres requisitos principales

- **Concurrencia** → Las tres tareas independientes a ser realizadas.
- **Medida de la duración del tiempo** → Muestreo de llaves y presión.
- **Cola de mensajes** → Manejo de colas de datos.

Estos son servicios típico de kernel de tiempo real

Un RTOS provee:

- Concurrencias, permitiendo crear tareas independientes.
- Posee una administración de tiempos.
- Posee una cola de mensajes para comunicarse entre tareas.

Ejemplo ahora en RTOS

Utilizando un RTOS, las funciones que se codificaron se transforman en tareas independientes.

Tarea para el manejo de los motores

```
void checkMotorSwitches(void) {  
    int i;  
    while(1) {  
        vTaskDelay(pdMS_TO_TICKS(100));  
        for (i=0; i<3; i++) {  
            if (isswitchON(i)) motorON(i);  
            else  
                motorOFF(i);  
        }  
    }  
}
```

Son bloques independiente con su propio bucle

Temporizaciones bloqueantes con funciones del SO

Funciones Bloqueantes en RTOS: No ocupa tiempo de CPU, la cual puede estar ejecutando otras tareas.

Ejemplo ahora en RTOS

Utilizando un RTOS, las funciones que se codificaron se transforman en tareas independientes.

Tarea para el manejo de Datagramas

Manejo de Colas con funciones del SO

```
void checkDatagramTask(void) {  
    typeMsg *msg;  
    while (1) {  
        if(xQueueReceive(xDatagramQueue, &msg, portMAX_DELAY) == pdPASS) {  
            processDataGram(msg);  
            vPortFree(msg);  
        }  
    }  
}
```

Ejemplo ahora en RTOS

Utilizando un RTOS, las funciones que se codificaron se transforman en tareas independientes.

Tarea para el manejo de las válvulas

```
void checkPressure(void) {  
    while(1) {  
        vTaskDelay(pdMS_TO_TICKS(50));  
        if (pressure( ) > 100) {  
            openValve( );  
            while(1) {  
                vTaskDelay(pdMS_TO_TICKS(50));  
                if (pressure( ) < 90) {  
                    closeValve( );  
                    break;  
                }  
            }  
        }  
    }  
}
```

Los procesos independientes,
permite solucionar problemas
de una manera mas sencilla



Ejemplo ahora en RTOS

Las funciones (ahora tareas) son lanzadas desde el main().

Funciones del kernel como `vtaskDelay()` o `xqueueReceive()` bloquea la tarea hasta que un evento determinado ocurra (en el ejemplo un tiempo o la llegada de un mensaje)

Utilizamos 3 funciones del kernel

- **`vtaskDelay()`** genera un retardo
- **`xqueueReceive()`** queda en estado de espera hasta recibir un mensaje
- **`vPortFree()`** libera el mensaje



Ejemplo ahora en RTOS

En el ejemplo en RTOS vemos entonces que

- La espera activa es eliminada.
- La administración del timer ya no es mas un asunto del programador.
- checkPressure no tiene que retener una variable de estado.
- El desarrollo y administración de colas ya no es mas un asunto del programador.

Ademas: RTOS permite asignar prioridades

Una tarea critica (verificar la presión), se le asigna una prioridad mas alta, esto permite que transcurrido los 50ms se verificará la presión interrumpiendo cualquier otra cosa que esté realizando el sistema.



¿Qué es un RTOS?

Una aplicación normalmente incluye una combinación de requisitos en tiempo real tanto duros como blandos.

- **Los requisitos blandos (soft)** son aquellos que establecen un plazo de tiempo, pero incumplir dicho plazo no inutilizaría el sistema.
- **Los requisitos estrictos (hard)** son aquellos que establecen un plazo, cuyo incumplimiento resultaría en un fallo absoluto del sistema.

RTOS (Sistema Operativo de Tiempo Real): Sistema operativo diseñado para aplicaciones que deben procesar datos y responder a eventos externos dentro de plazos de tiempo estrictos y predecibles (requisitos hard).



¿Qué es un RTOS?

El RTOS gestiona el tiempo y los recursos del procesador.

- **Tareas (Tasks):** La aplicación se divide en "mini-programas" independientes que se ejecutan de forma concurrente.
- **Scheduler (Planificador):** Decide qué tarea ejecutar en cada instante, basándose en su prioridad y tiempo de inactividad.
- **Multitarea:** Aunque solo una tarea se ejecuta a la vez en un único núcleo, el cambio rápido entre ellas crea la ilusión de que se ejecutan en paralelo.



¿Qué Ganamos con FreeRTOS?

- **Modularidad:** Las tareas son módulos independientes.
- **Eficiencia:** El código es dirigido por eventos. El procesador solo trabaja cuando es necesario, ahorrando energía.
- **Mantenibilidad:** Es más fácil añadir o quitar funcionalidades sin afectar al resto del sistema.
- **Reutilización de Código:** Mayor modularidad permite reutilizar código.
- **Abstracción:** Nos abstrae de los detalles de bajo nivel de la temporización, simplificando el diseño.
- **Desarrollo en equipos:** Las tareas tienen interfaces definidas, permite desarrollo en equipos de trabajo
- **Depuración sencilla:** las tareas independiente facilita el debug



¿ Preguntas ?